UNIVERSITÀ DEGLI STUDI DI MILANO-BICOCCA
Dipartimento di Informatica, Sistemistica e Comunicazione

A Language-Agnostic Framework for Dependency Extraction

Master's Thesis in Computer Science



Candidate
Christian Ferrario

Supervisors
Prof. Francesca Arcelli Fontana
Dr. Francesco Refolli

Academic Year 2025–2026

1

Context & Motivation

1 Context: Software Evolution & Dependency Analysis

Technical Debt and the Role of Dependency Graphs

EVOLUTION & DECAY

Architectural Technical Debt
Continuous changes degrade system architecture and accumulate structural debt over time.

THE POLYGLOT REALITY

Heterogeneous Systems
Production codebases routinely combine multiple languages and runtime environments.

THE ESSENTIAL NEED

Structural Visibility
Reliable dependency graphs are required to detect decay and govern system architecture.

Why Dependency Graphs?

Detect Architectural Smells
Uncover cyclic dependencies, unstable interfaces, and modular erosion beyond single-file scope.

Safe Refactoring & Impact Analysis
Trace change blast radius and ripple effects before altering critical components.

System Comprehension
Provide an accurate structural map to navigate and understand complex codebases.

COMPILER COUPLING

Tied to Native Front-Ends

Analyzers rely on compiler internals forcing extraction pipelines to be re-engineered for each language.

ENGINEERING BURDEN

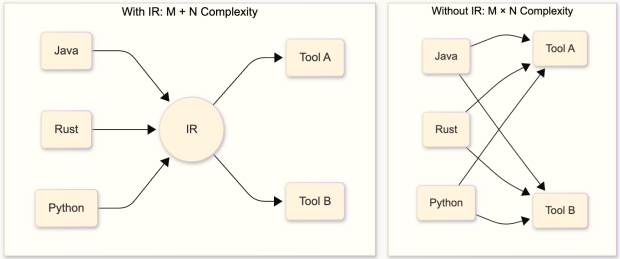
The $M \times N$ Proliferation

Supporting new languages requires adapting and bundling toolchains, duplicating engineering efforts.

THE CORE GOAL

Decoupled Analysis Pipeline

A single, language-agnostic extraction flow replacing fragmented pipelines.



Tool proliferation ($M \times N$) vs. unified intermediate representation ($M + N$)

1 Structural Convergence of Syntax Trees

Syntactic Homology Across Language Grammars

GENERIC PARSING

Tree-sitter CSTs

Fast, incremental parser generation producing concrete syntax trees from formal grammars.

PRODUCTION ECOSYSTEM

Universal Coverage

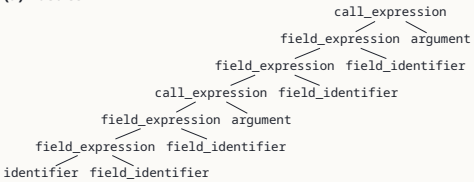
Broad open-source adoption covering virtually all programming languages used in production.

STRUCTURAL CONVERGENCE

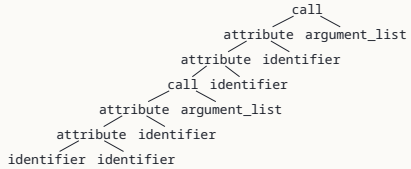
Syntactic Homology

Shared grammar design patterns lead equivalent constructs to yield isomorphic syntax subtrees.

(a) Rust CST



(b) Python CST



Structural convergence across languages for the code `a.b.c().d.e()`.

Research Objectives & OmniDeps Proposal

Design Principles and Evaluation Targets

Design, formalize, and implement **OmniDeps**: an extensible, language-agnostic static analysis framework for extracting architectural dependency graphs.

Pipeline Formalization

Functional Staging
Type-safe 3-phase composition
Universal IR
Language-agnostic domain model

Rust Implementation

Decoupled Engine
Zero compiler dependencies
Query Algebra
Formal scope navigation

Empirical Evaluation

Graph Harmonization
Cross-tool ontological alignment
Large-Scale Benchmark
156 repos (∼98M LOC) vs SOTA

Java C / C++ Python Rust — Syntactically and semantically diverse languages evaluated to validate language-agnostic extensibility

2
The Framework

Analysis Pipeline Overview

Functional Composition and Type-Safe Staging

$$\phi : \mathcal{W} \to \mathcal{G}, \qquad \phi = \gamma \circ \rho \circ \varepsilon = \gamma \circ (\rho_{\text{exec}} \circ \rho_{\text{build}}) \circ \varepsilon$$

PHASE 1: EXTRACTION

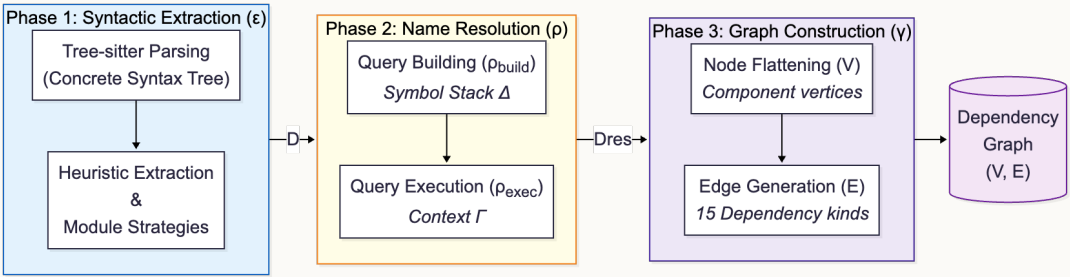
$$\varepsilon : \mathcal{W} \to \mathcal{D}$$

PHASE 2: RESOLUTION

$$\rho : \mathcal{D} \to \mathcal{D}_{\text{res}}$$

PHASE 3: GRAPH CONSTRUCTION

$$\gamma : \mathcal{D}_{\text{res}} \to \mathcal{G}$$



2 Phase 1: Syntactic Extraction

From Concrete Syntax Trees to Intermediate Representation

CST TRAVERSAL

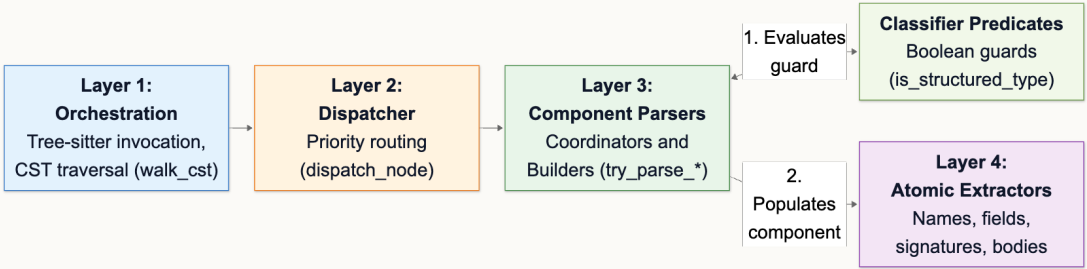
Tree-sitter Parsing
Systematic AST visitor across files

HEURISTIC RECOGNITION

Declarative Guards
Boolean rules for structural entities

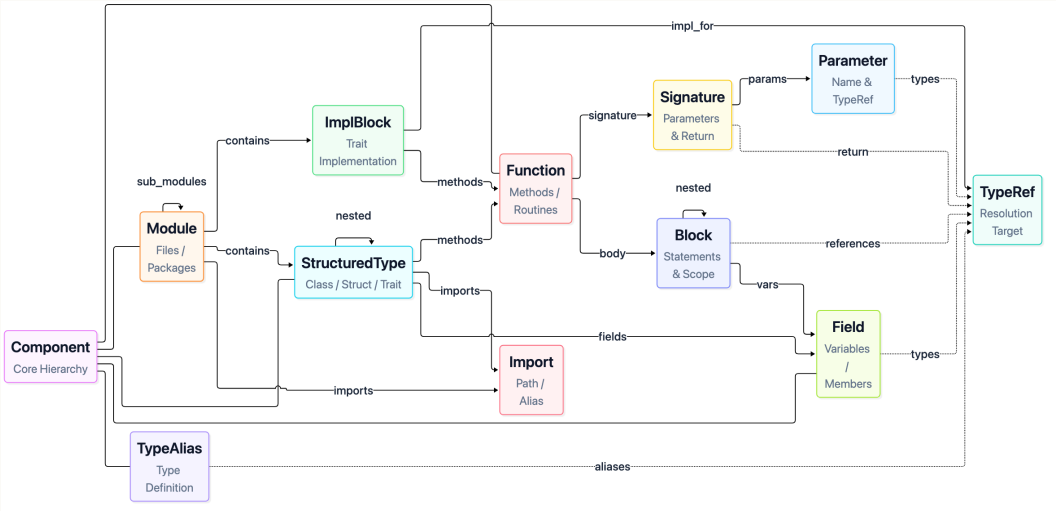
IR INSTANTIATION

Symbol Populating
Direct extraction into unlinked IR



2 Universal Intermediate Representation (IR)

Hierarchical Domain Model and Language Abstraction



2 Phase 2: Decoupled Name Resolution

TypeRef Binding and Two-Step Scope Navigation

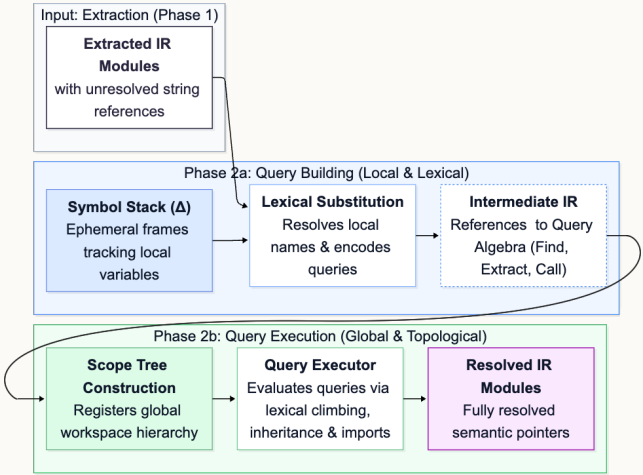
TWO-STEP RESOLUTION

1. Query Building (ρ_{build})
Local stack & closed queries

2. Query Execution (ρ_{exec})
Deterministic Scope Tree navigation

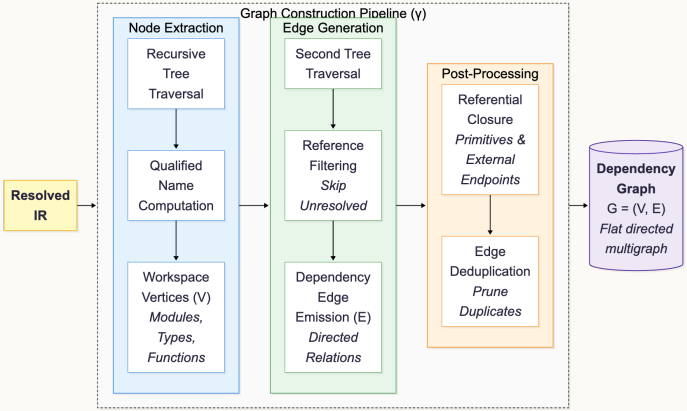
QUERY PRIMITIVES

- **Find** Lexical climbing
- **Extract** Member descent
- **Call** Return propagation

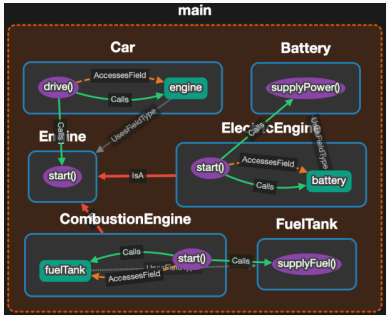


2 Phase 3: Graph Construction & Projection

From Resolved IR Trees to Directed Dependency Multigraphs



RESULTING DEPENDENCY GRAPH



3
Experimental Validation

3 Experimental Setup

Research Questions and Background Truth

RESEARCH QUESTIONS

RQ1: Baseline construct coverage on synthetic oracles

RQ2: Cross-tool structural alignment (156 repos, 5 languages)

RQ3: Execution throughput, speedup, and scalability

BASELINE TOOL LANGUAGE COVERAGE

Tool	C	C++	Java	Python	Rust
OmniDeps	✓	✓	✓	✓	✓
Depends	✓	✓	✓	✓	—
Arcan	✓	✓	✓	×	—
Entangle	—	—	✓	—	×

RQ1: SYNTHETIC ORACLES

Dedicated benchmark suites with YAML architectural manifests testing constructs in isolation.

RQ1 Results:

Successfully validated against all five language benchmarks, showing the baseline soundness of the approach.

SCOPE LIMITATION

- Verifies query correctness in a controlled setting.
- **Does not guarantee generalization** to real-world codebases.
- Motivates comparative evaluation against established analyzers.

3 Cross-Tool Methodology & Feasibility

Graph Harmonization, Similarity Metrics, and Analysis Completion

METAMODEL HETEROGENEITY

Direct intersection fails due to representation disparities across tools:

- **Granularity:** Members (methods) vs. types (classes)
- **Taxonomy:** Divergent relation vocabularies
- **Naming:** File paths vs. fully qualified identifiers

GRAPH HARMONIZATION (GRAPH-SYNCHRON)

Automated normalization pipeline:

- **Path Normalization** & canonical package synthesis
- **Member Lifting** to enclosing type
- **Taxonomy Alignment**

EXECUTION COMPLETION ON 156 REPOSITORIES

Language	OmniDeps	Depends	Arcan	Entangle
C (31)	31 (100%)	30 (97%)	27 (87%)	—
C++ (31)	30 (97%)	25 (81%)	16 (52%)	—
Java (32)	32 (100%)	31 (97%)	18 (56%)	19 (59%)
Python (30)	30 (100%)	29 (97%)	0 (0%)	—
Rust (32)	32 (100%)	—	—	0 (0%)
Total (156)	155 (99.4%)	115 (93%)	61 (49%)	19 (30%)

GRAPH COMPARISON METRICS

$$J(A,B) = \frac{|A \cap B| + 1}{|A \cup B| + 1} \quad \text{(Smoothed Jaccard)}$$

$$\text{Inf}(A,B) = \frac{|B \setminus A|}{|A \cap B|} \quad \text{(Omission)}$$

$$\text{Sup}(A,B) = \frac{|A \setminus B|}{|A \cap B|} \quad \text{(Novelty)}$$

3 RQ2: Cross-Tool Structural Alignment

Object-Oriented Parity, Dynamic Language

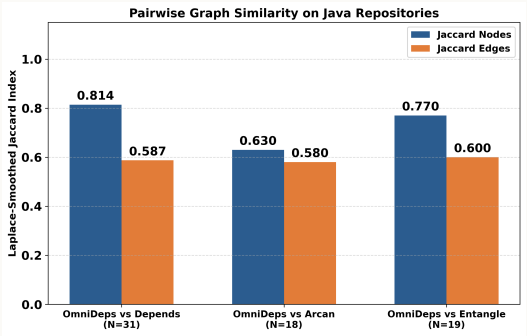
CROSS-TOOL AGREEMENT ACROSS PARADIGMS

Lang	Baseline	J_{nodes}	Omission	Novelty	J_{edges}
Java	Depends	0.814	2.1%	28.6%	0.587
	Entangle	0.770	19.6%	19.9%	0.600
	Arcan	0.630	0.3%	79.6%	0.580
Python	Depends	0.852	4.8%	16.1%	0.370
C / C++	*Detailed on next slide (preprocessor dynamics)*				

EMPIRICAL FINDINGS

- **OOP Parity (Java):** Near-identical topology without JVM or bytecode.
- **Dynamic Lifting (Python):** Fixture harmonization cuts omission to 4.8%.
- **Polyglot Workspaces:** Compiler-based tools only analyze given language, Omnideps scans all files in a workspace, e.g. Rust core in Pydantic.

PAIRWISE SIMILARITY IN JAVA



Smoothed Jaccard indices across Java repos vs Depends, Arcan, Entangle

3 RQ2: Structural Alignment in C & C++

The Preprocessor Dilemma and Ontological Disparity

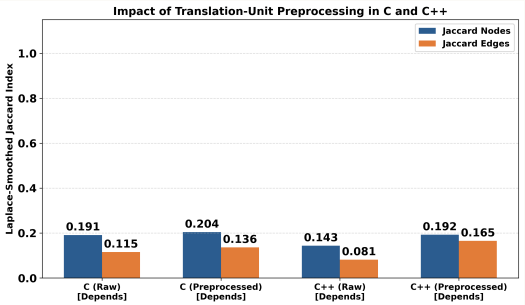
CROSS-TOOL AGREEMENT IN C & C++

Lang	Baseline / Mode	J_{nodes}	J_{edges}	Outcome
C / C++	Arcan (File-level)	≤ 0.01	≤ 0.00	—
C	Depends (Raw)	0.191	0.115	Baseline
	Depends (Preproc.)	0.204	0.136	+18% Edges
C++	Depends (Raw)	0.143	0.081	Baseline
	Depends (Preproc.)	0.192	0.165	2× Recall
C (sds)	Depends	1.000	1.000	Perfect Parity

EMPIRICAL FINDINGS

- **Ontological Disparity:** Arcan tracks physical files ($J \leq 0.01$); OmniDeps extracts logical architectural entities.
- **Preprocessor Dilemma:** Macro expansion doubles edge recall ($+104\%$), but duplicates header types across translation units.
- **Modular Soundness:** Perfect agreement ($J = 1.0$) on self-contained modular libraries like `antirez/sds`.

IMPACT OF MACRO PREPROCESSING



Similarity before and after macro expansion vs Depends

3 RQ3: Runtime Performance and Scalability

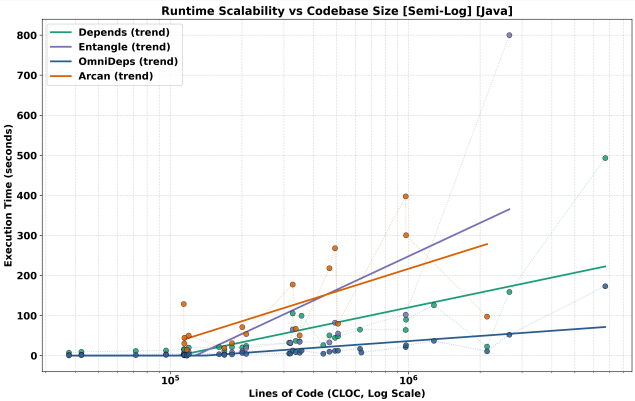
Linear Scaling and Throughput for Continuous Auditing

COMPUTATIONAL EFFICIENCY

- **Strict $O(N)$ Scaling:** time complexity scales linearly with the codebase size.
- **High Throughput:** Sustained **74k+ LOC/s**, outperforming compiler-coupled baselines.
- **Seconds on 1M+ LOC:** Fast analysis even of large codebases.
- **Continuous Auditing:** Practical for real-time and continuous checks.

$O(N)$ Linear | CI/CD Ready

EXECUTION TIME VS. CODEBASE SIZE



Wall-clock extraction time vs. codebase size (LOC) across Java repositories

3 Conclusions & Outlook

Viability of Language-Agnostic Extraction and Future Research Directions

Language-agnostic, syntax-directed extraction proves to be a **viable and sound approach** for architectural analysis: it recovers reliable dependency graphs in seconds without compiler friction.

THREATS TO VALIDITY

- **Oracle Absence:** Measures cross-tool concordance against mature baselines rather than absolute ground truth.
- **Harmonization Level:** Evaluates macro-architectural (type) convergence, abstracting statement-level details.
- **C/C++ Preprocessing:** Trade-off between raw CST macro masking and translation-unit header duplication.

FUTURE RESEARCH DIRECTIONS

- **Linker Deduplication:** Reconciliation pass to merge duplicated header symbols across C/C++ translation units.
- **Non-Imperative Paradigms:** Extending query algebra to functional (Haskell) and declarative logic models.
- **Cross-Language FFI:** Tracking foreign function interface boundaries (FFI/JNI) via unified Scope Trees.

UNIVERSITÀ DEGLI STUDI DI MILANO-BICOCCA · DISCo

Thanks for your attention!

A Language-Agnostic Framework for Dependency Extraction

CANDIDATE

Christian Ferrario

SUPERVISORS

Prof. Francesca Arcelli Fontana
Supervisor

Dr. Francesco Refolli
Co-Supervisor

Questions and discussion are welcome

Replicability Package

Zenodo DOI: 10.5281/zenodo.23315197

github.com/FerrarioChristian/omnideps

3 Appendix: Language Configuration Profiles ($\mathcal{K}$)

Declarative Language Product Line Parameters Across Supported Ecosystems

1. MODULE ORGANIZATION STRATEGIES (MODULECONFIG)

Lang	File	Dir	PkgDecl	Namespace	InlineMod
Java	—	—	✓	—	—
Rust	✓	✓	—	—	✓
Python	✓	✓	—	—	—
C++	—	—	—	✓	—
C	—	—	—	—	—

C defaults to flat root module; C++ uses open namespaces; Rust uses closed inline mods.

2. DECLARATION LINKING & VISIBILITY

Lang	TransImports	AssignAliases	CallableTypes
Java	—	—	—
Rust	—	—	—
Python	✓	✓	✓
C++	—	—	✓
C	—	—	—

Python enables transitive re-exports (`__init__.py`) and assignment-based TypeVar.

3. INSTANCE SEMANTICS & RECEIVER RESOLUTION

Lang	SelfKey	ImplicitSelf	DynFields	SelfType
Java	`this`	—	—	—
Rust	`self`	—	—	`Self`
Python	—	✓	✓	—
C++	`this`	—	—	—
C	—	—	—	—

Python binds 1st param to instance and harvests fields in `__init__`; Rust injects `Self`.

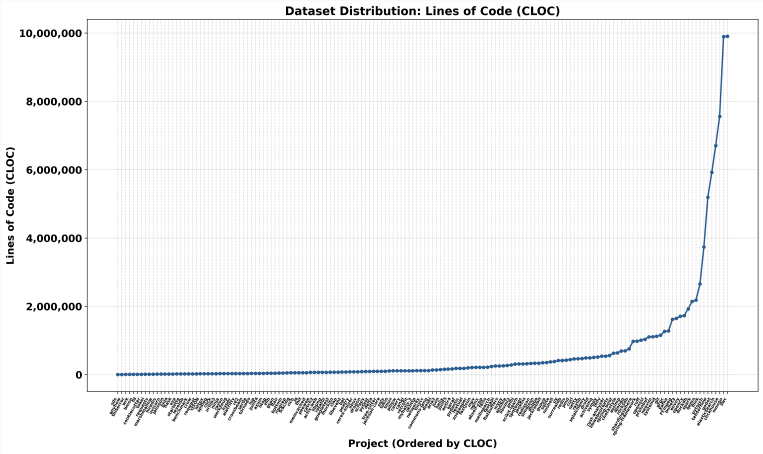
LANGUAGE PRODUCT LINE (LPL) TUPLE

$\mathcal{K} = \langle \text{ModuleConfig}, \text{TransitiveImports}, \text{AssignmentAliases}, \ldots \rangle$

- **Agnostic Engine:** The resolution and graph construction phases never branch on hardcoded language names.
- **Declarative Extension:** Adding a language requires only declaring its feature profile $\mathcal{K}_L$.

3 Appendix: Dataset Distributions

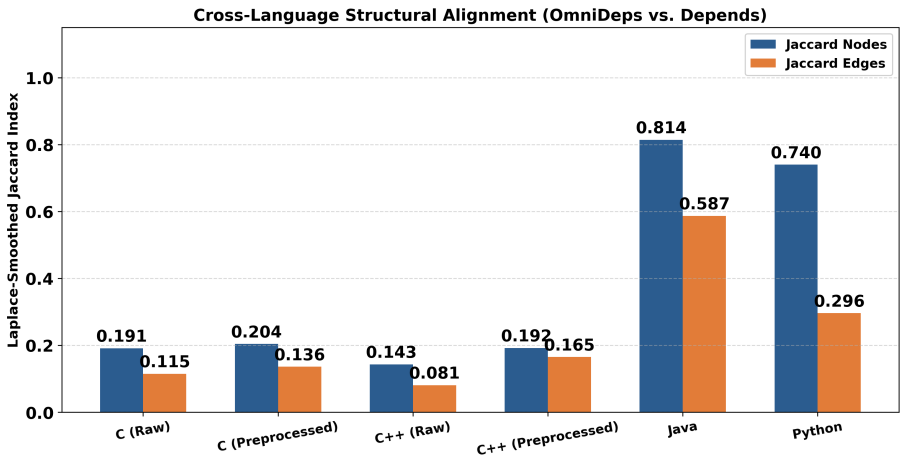
Code Density (LOC/Files) and Pure Codebase Size (CLOC) across 156 Repositories



Pure codebase size: Lines of Code (CLOC) ordered ascending

3 Appendix: Cross-Language Graph Similarity

Pairwise Laplace-Smoothed Jaccard Index Across All Analyzed Tool Pairs



Mean Laplace-smoothed Jaccard similarity for nodes and edges across all comparative tool pairs

3 Appendix: Pairwise Agreement & Discrepancy Matrix

Full Cross-Tool Evaluation Results Across All Comparators ($N = 156$)

Target Pair	N	J_{nodes}	Sup_n	Inf_n	J_{edges}	Sup_e	Inf_e
Java (vs. Depends)	31	0.814 (0.849)	0.286	0.021	0.587 (0.609)	0.406	0.345
Java (vs. Entangle)	19	0.770 (0.782)	0.199	0.196	0.600 (0.598)	0.359	0.366
Java (vs. Arcan)	18	0.630 (0.693)	0.796	0.003	0.580 (0.575)	0.759	0.092
Python (vs. Depends)	29	0.852 (0.882)	0.161	0.048	0.323 (0.300)	1.474	0.805
Python (No Imports)	29	0.852 (0.882)	0.161	0.048	0.368 (0.387)	1.017	0.834
C Raw (vs. Depends)	30	0.191 (0.167)	14.017	0.619	0.115 (0.042)	44.307	1.636
C Preproc (vs. Depends)	31	0.204 (0.158)	7.218	0.733	0.136 (0.083)	37.777	3.993
C++ Raw (vs. Depends)	25	0.143 (0.127)	12.493	2.969	0.081 (0.039)	58.836	4.769
C++ Preproc (vs. Depends)	22	0.192 (0.151)	4.911	3.229	0.165 (0.083)	9.962	8.320

Values report arithmetic means (medians in parentheses). Sup $= |A \setminus B| / |A \cap B|$, Inf $= |B \setminus A| / |A \cap B|$. Arcan in C/C++ omitted due to ontological disparity (file vs. type).

3 Appendix: Multi-Language Runtime Scalability

Wall-Clock Extraction Time vs. Codebase Size (LOC) on Semi-Log Scale

